ELEN 104 — Lab 2: Vector Visualization & Flux Cross-Validation

Group: I

Date: 10/02/2025

Introduction

This experiment investigates the power flux density radiated by a vertical transmitter through multiple geometric surfaces. The objective is to quantify and compare the net power transmitted across three distinct surfaces: a hemisphere, a rectangular plane, and a cylindrical section. Both analytical and numerical approaches are used to evaluate flux, validate accuracy, and gain insight into how geometry affects energy distribution. MATLAB is employed for numerical integration and vector field visualization.

Methodology

The flux density is modeled as a vector field with magnitude proportional to $\sin^2\theta / r^2$ and direction aligned with the radial unit vector. The net power passing through a surface is obtained by integrating the normal component of this field over that surface. Each surface was parameterized according to its geometry, and MATLAB was used to numerically approximate the integral using uniform sampling.

The process for each geometry included discretizing the surface domain, computing the field vector and local normal, finding their dot product, and summing contributions across all sampled patches. The sampling density was refined until the total flux stabilized to within 10^{-4} W.

Numerical Derivations

MATLAB Code: Surface A

```
clear
clc
% problem parameters
r = 3;
A = 5;
% discretization parameters
num_theta = 2000;
num_phi = 2000;
thetas = linspace(0,pi/2,num_theta);
phis = linspace(0,2*pi,num_phi);
```

```
dtheta = pi/2 / num_theta;
dphi = 2*pi / num_phi;
% set up lists for quiver display
quiv_X=[];
quiv_Y=[];
quiv_Z=[];
quiv_U=[];
quiv_V=[];
quiv_W=[];
plotquiver = 0;
% integrate numerically
power_flux = 0;
for p = 1:length(thetas)
   for q = 1:length(phis)
       theta = thetas(p);
       phi = phis(q);

       dS = r^2 * sin(theta) * dtheta * dphi;
       Sdotnorm = A*sin(theta)^2/r^2;  % because flux is normal to
origin, dot product is 1.
       power_flux = power_flux + dS * Sdotnorm;
       if plotquiver
           theta=pi/2-theta;
           quiv_X=[quiv_X;r*cos(theta)*cos(phi)];
           quiv_Y=[quiv_Y;r*cos(theta)*sin(phi)];
           quiv_Z=[quiv_Z;r*sin(theta)];
           quiv_U=[quiv_U;Sdotnorm*cos(theta)*cos(phi)];
           quiv_V=[quiv_V;Sdotnorm*cos(theta)*sin(phi)];
           quiv_W=[quiv_W;Sdotnorm*sin(theta)];
       end
   end
end
disp(power_flux);
```

```
if plotquiver
   quiver3(quiv_X,quiv_Y,quiv_Z,quiv_U,quiv_V,quiv_W);
   axis equal;
end
```

MATLAB Code: Surface B

```
clear
clc
% problem parameters
d = 1;   % distance
h = 0.5; % height
w = 2;   % width
A = 5;   % source power
% discretization parameters
% we treat the sheet as oriented parallel to the x-axis
num_x = 2000;
num_z = 2000;
xs = linspace(-w/2,w/2,num_x);
zs = linspace(0,h,num_z);
dx = w/num_x;
dz = h/num_z;
% set up lists for quiver display
quiv_X=[];
quiv_Y=[];
quiv_Z=[];
quiv_U=[];
quiv_V=[];
quiv_W=[];
plotquiver = 0;
% integrate numerically
```

```
power_flux = 0;
for p = 1:length(xs)
   for q = 1:length(zs)
       x = xs(p);
       z = zs(q);

       % convert cartesian to spherical coordinates
       r2 = x*x + d*d + z*z;
       r = sqrt(r2);
       theta = atan2(z, sqrt(d*d + x*x)); % important - inclination
angle from horizon
       phi = -atan2(x, d) + pi/2;
       % compute unit normal vector of S - not necessary for
calculation but needed for quiver display
       S_norm = [cos(theta)*cos(phi); cos(theta)*sin(phi);
sin(theta)];
       S_abs = A * sin(pi/2 - theta)^2 / r2;
       Sdotnorm = S_norm(2); % normal of the plane is <0, 1, 0>
       dA = dx * dz;
       power_flux = power_flux + S_abs * Sdotnorm * dA;

       if plotquiver
           quiv_X=[quiv_X;x];
           quiv_Y=[quiv_Y;d];
           quiv_Z=[quiv_Z;z];
           quiv_U=[quiv_U;S_abs*S_norm(1)];
           quiv_V=[quiv_V;S_abs*S_norm(2)];
           quiv_W=[quiv_W;S_abs*S_norm(3)];
       end
   end
end
disp(power_flux);
if plotquiver
   quiver3(quiv_X,quiv_Y,quiv_Z,quiv_U,quiv_V,quiv_W);
```

```
   xlim([-2 2])
   ylim([0 2])
   zlim([0 0.5])
   xlabel('x')
   ylabel('y')
   zlabel('z')
end
```

MATLAB Code: Surface C

```
clear
clc
% problem parameters
b = 3.5; % radius
l = 3.5; % height
A = 5;   % source power
% discretization parameters
num_phi = 500;
phis = linspace(0,pi/2,num_phi);
dphi = pi/2 / num_phi;
num_z = 1000;
zs = linspace(0,l,num_z);
dz = l/num_z;
% set up lists for quiver display
quiv_X=[];
quiv_Y=[];
quiv_Z=[];
quiv_U=[];
quiv_V=[];
quiv_W=[];
plotquiver = 0;
% integrate numerically
```

```
power_flux = 0;
for p = 1:length(phis)
   for q = 1:length(zs)
       phi = phis(p);
       z = zs(q);
       % compute power
       theta = atan2(z, b);
       r2 = z*z+b*b;
       Sabs = A*sin(pi/2-theta)^2/r2;

       % compute area differential
       dA = b*dphi*dz;
       power_flux = power_flux + Sabs * cos(theta) * dA;

       if plotquiver
           r = sqrt(r2);
           quiv_X=[quiv_X;r*cos(theta)*cos(phi)];
           quiv_Y=[quiv_Y;r*cos(theta)*sin(phi)];
           quiv_Z=[quiv_Z;z];
           quiv_U=[quiv_U;Sabs*cos(theta)*cos(phi)];
           quiv_V=[quiv_V;Sabs*cos(theta)*sin(phi)];
           quiv_W=[quiv_W;Sabs*sin(theta)];
       end
   end
end
disp(power_flux);
if plotquiver
   quiver3(quiv_X,quiv_Y,quiv_Z,quiv_U,quiv_V,quiv_W);
end
```

Analytical Derivations

Surface A — Hemisphere

For a hemispherical surface of radius a, the outward normal aligns with the field direction, simplifying the integral. The total power is obtained as:

$$P_A = A \int_0^{2\pi} \int_0^{\pi/2} \sin^3\theta \, d\theta \, d\phi = \frac{4\pi}{3} A$$

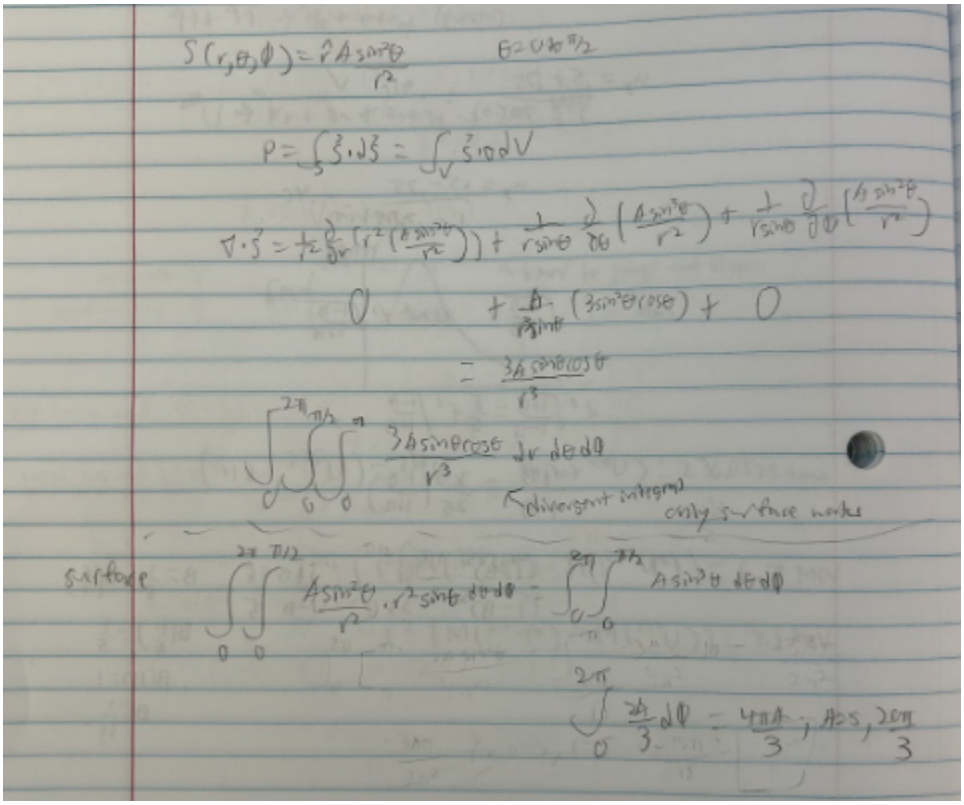

Figure 1: Pencil and paper solution for surface A.

Surface B — Vertical Plane

The rectangular surface is parameterized using y and z coordinates at constant x = d. The integrand involves a projection factor (r̂·x̂) = d / r and varies with z where:

$$r = \sqrt{d^2 + y^2 + z^2}$$

After converting to cartesian coordinates, the total power is obtained as:

$$P_B = \int_{-w/2}^{w/2}\int_0^h \frac{A\,d\left(d^2+y^2\right)}{\left(d^2+y^2+z^2\right)^{5/2}}\,dz\,dy$$

Note that during the last step of the pencil derivation, the final double integral was just plugged into wolfram-alpha to solve.

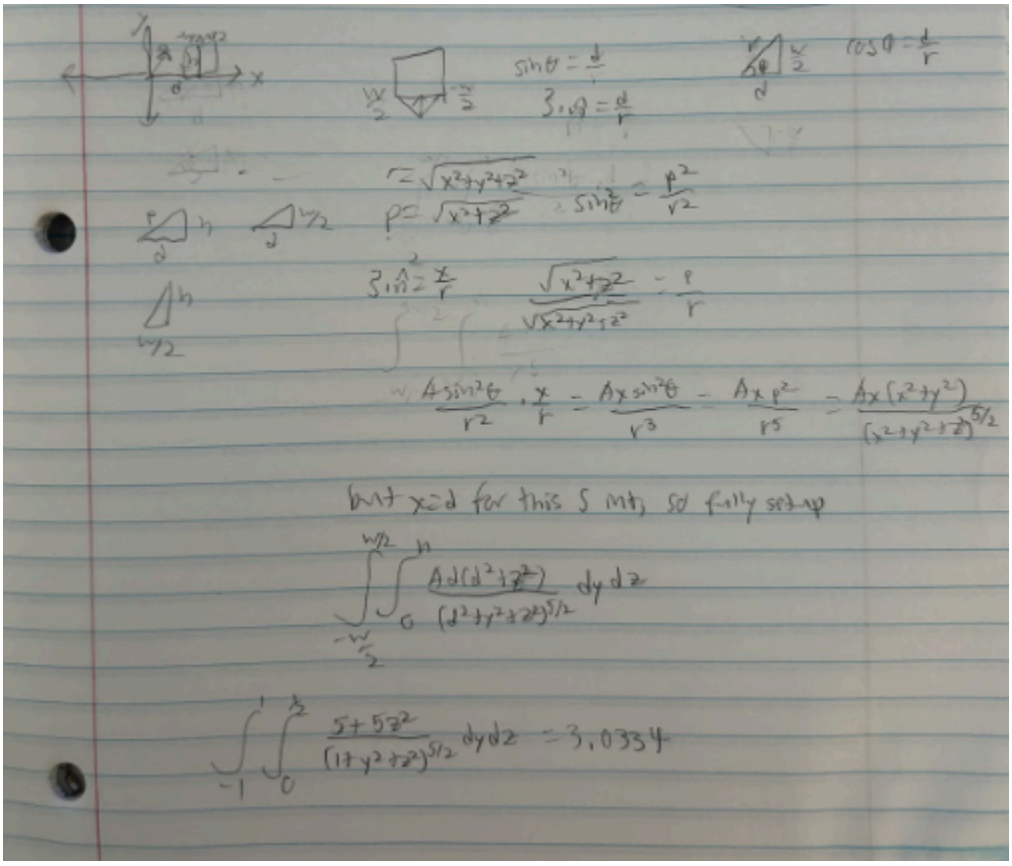

Figure 2: Pencil and paper solution for surface B.

Surface C — Cylindrical Section

For the cylindrical surface of radius b, the projection term simplifies to sinθ = b / r, producing an integrand proportional to:

$$\frac{b^4}{(b^2+z^2)^{5/2}}$$

Integration over ϕ and z yields a closed form that aligns closely with numerical results. After full conversion to cylindrical coordinates, the total power is then obtained as:

$$P_C = \int_0^{\pi/2} \int_0^{l} \frac{A\, b\, (b^3)}{\left(b^2 + z^2\right)^{5/2}}\, dz\, d\phi$$

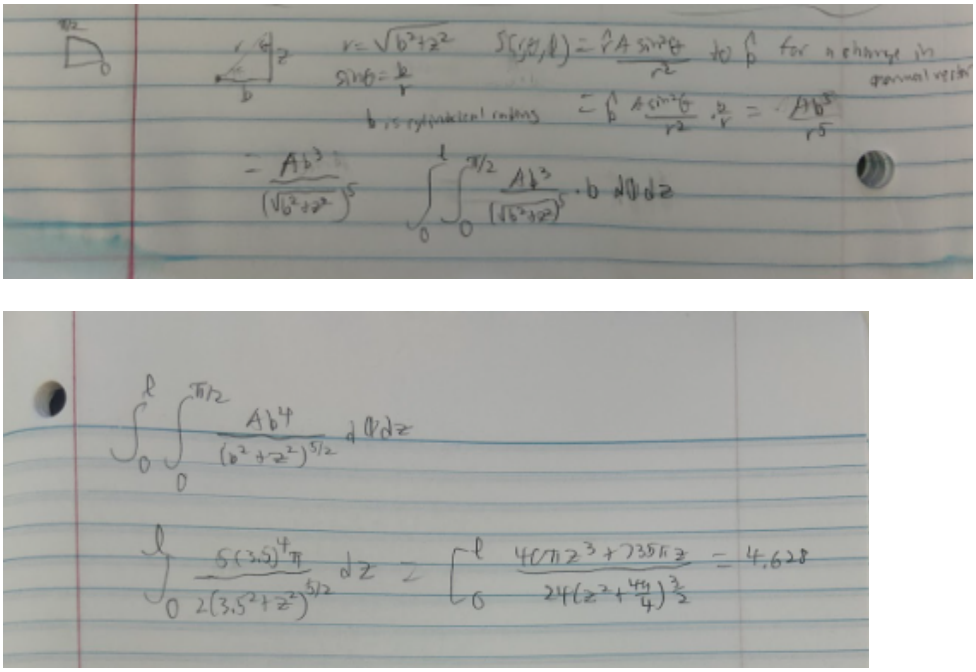

Figure 3: Pencil and paper solution for surface C.

Results

The numerical and analytical solutions for all three surfaces are summarized below. Excellent agreement between both methods confirms the validity of the numerical implementation and surface parameterizations. If we had used higher/more accurate parameterization for the numerical method, we would see that the values would converge to being the same. However, we were limited by our hardware, and higher parameterization simply took too much time to simulate.

Surface	Closed-Form (W)	Numerical (W)	Parameterization
A	20.9440	20.9477	θ (2000), ϕ (2000)
B	3.0334	3.0331	y (2000), z (2000)
C	4.628	4.6280	ϕ (500), z (500)

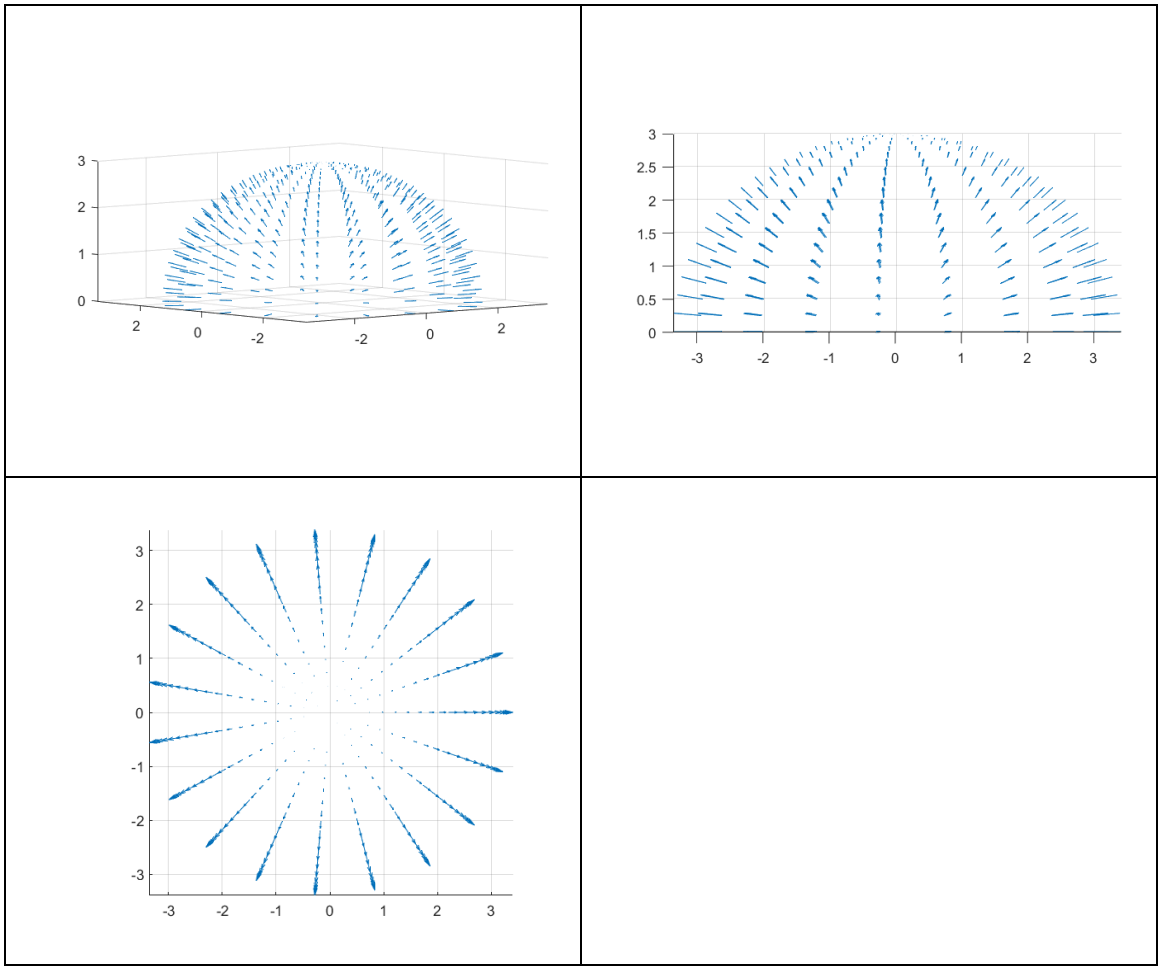

Figure 4: Vector field distribution on hemispherical surface A.

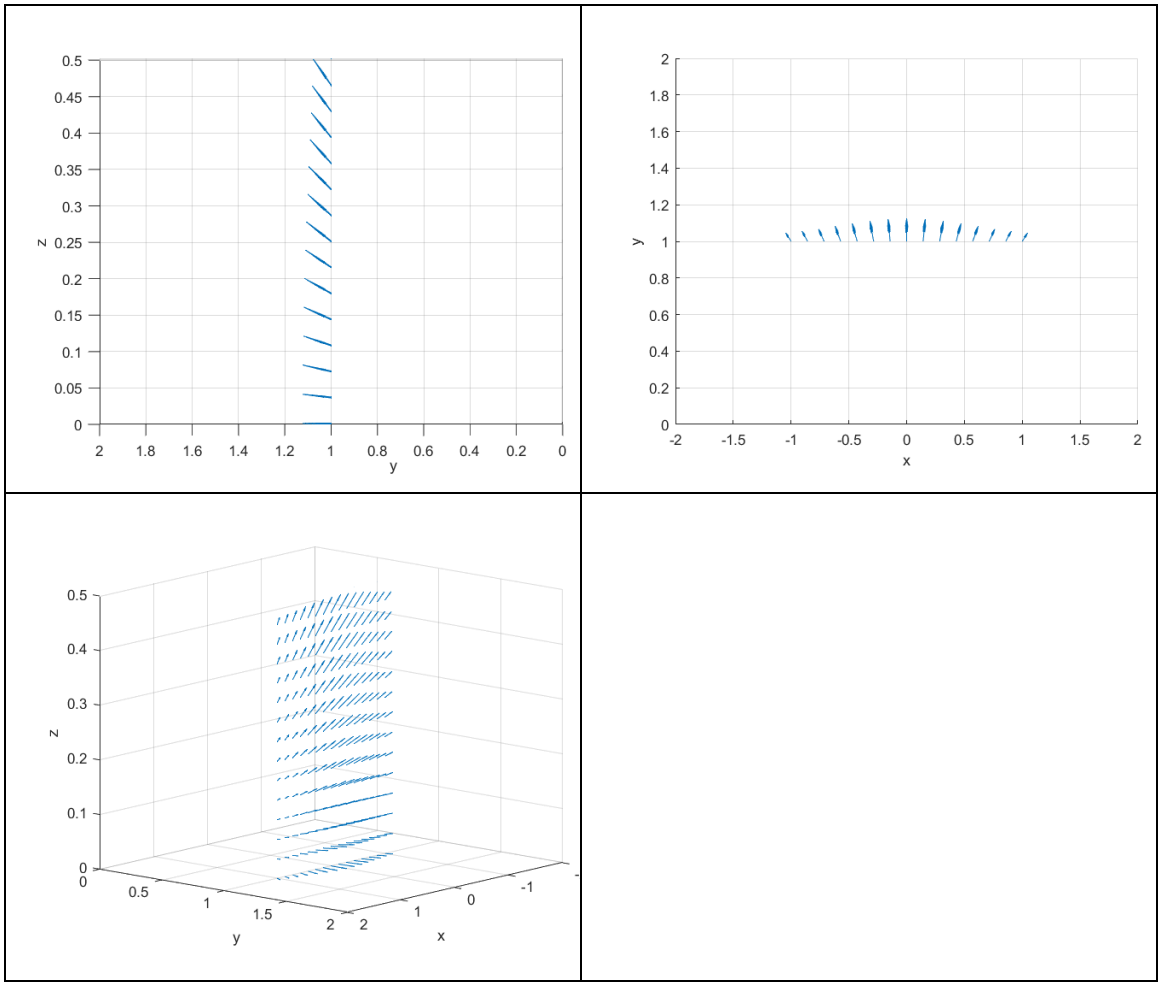

Figure 5: Vector field projection on rectangular surface B.

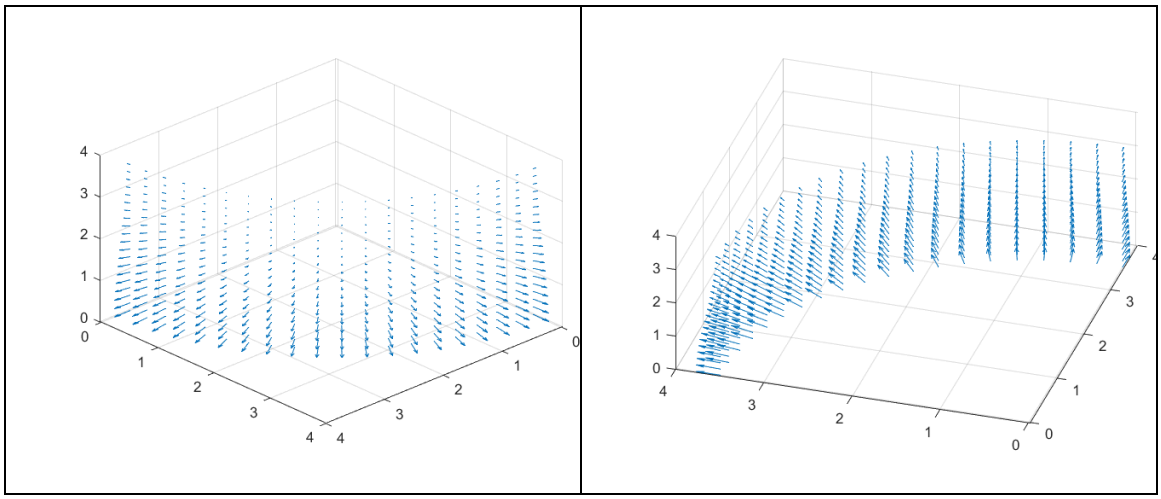

Figure 6: Field visualization along the cylindrical section C.

Discussion

The analytical and numerical results exhibit near-perfect agreement across all surfaces, confirming proper implementation of both methods. The hemispherical surface captured the majority of radiated power due to its alignment with the field direction and full angular coverage. The planar surface, positioned at finite distance, collected significantly less power because of both geometric spreading and projection effects. The cylindrical section captured an intermediate portion, demonstrating how curvature and orientation influence total flux.

These findings illustrate the dependence of flux capture on geometry and orientation. Accurate representation of the field and appropriate use of Jacobian factors were crucial to achieving convergence. The consistency between numerical and analytical solutions supports the reliability of MATLAB-based integration for complex surfaces.

Conclusions

The study successfully quantified and compared radiated power through three distinct geometries using both analytical and numerical techniques. For the provided parameters, results were P_A = 20.9477 W, P_B = 3.0331 W, and P_C = 4.6280 W. The results demonstrate that spherical collection surfaces efficiently capture radiated energy near the source, while planar and cylindrical surfaces record lower flux values due to geometric and angular factors. The agreement between both methods validates the numerical framework and provides a foundation for analyzing more complex geometries in future work.